

Лекція 15

Веб-застосунок на NestJs. Робота з БД (MongoDB). Тестування.



Nest JS

Getting started

Nest CLI:

```
$ npm i -g @nestjs/cli
```

```
$ nest new project-name
```

Alternatively, to install the TypeScript starter project with Git:

```
$ git clone https://github.com/nestjs/typescript-starter.git project
```

```
$ cd project
```

```
$ npm install
```

```
$ npm run start
```

Getting started

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}

bootstrap();
```

Getting started

Nest прагне бути платформо-агностичним фреймворком. Незалежність від платформи дозволяє створювати повторно використовувані логічні частини, які розробники можуть використовувати в кількох різних типах програм.

Технічно **Nest** може працювати з будь-яким HTTP-фреймворком Node після створення адаптера. Існують дві HTTP-платформи, що підтримуються "з коробки": **express** та **fastify**. Ви можете вибрати ту, яка найкраще відповідає вашим потребам.

platform-express - Express — це добре відомий мінімалістичний веб-фреймворк для Node. Це перевірена в бойових умовах, готова до використання бібліотека з безліччю ресурсів, реалізованих спільнотою. Пакет **@nestjs/platform-express** використовується за **замовчуванням**. Багато користувачів добре працюють з Express і їм не потрібно нічого робити, щоб його ввімкнути.

platform-fastify - Fastify — це високопродуктивний фреймворк з низькими накладними витратами, що зосереджений на забезпеченні максимальної ефективності та швидкості. Дізнайтеся, як ним користуватися, тут: <https://docs.nestjs.com/techniques/performance>

Routing

```
import { Controller, Get } from '@nestjs/common';

@Controller('cats')
export class CatsController {
  @Get()
  findAll(): string {
    return 'This action returns all cats';
  }
}
```

It's that simple. Nest provides decorators for all of the standard HTTP methods:

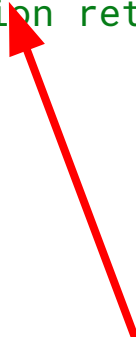
`@Get()`, `@Post()`, `@Put()`, `@Delete()`, `@Patch()`, `@Options()`, and `@Head()`.

In addition, `@All()` defines an endpoint that handles all of them.

Routing

```
import { Controller, Get, Req } from '@nestjs/common';  
import type { Request } from 'express';
```

```
@Controller('cats')  
export class CatsController {  
  @Get()  
  findAll(@Req() request: Request): string {  
    return 'This action returns all cats';  
  }  
}
```



```
app.get('/user/:id', (request, response) => {  
  response.send(`user ${request.params.id}`)  
})
```

Routing

@Request(), @Req()

req

@Response(), @Res()

res

@Next()

next

@Session()

req.session

@Param(key?: string)

req.params / req.params[key]

@Body(key?: string)

req.body / req.body[key]

@Query(key?: string)

req.query / req.query[key]

@Headers(name?: string)

req.headers / req.headers[name]

@Ip()

req.ip

@HostParam()

req.hosts

Routing

Маршрути на основі шаблонів також підтримуються в NestJS. Наприклад, зірочку (*) можна використовувати як символ підстановки для зіставлення будь-якої комбінації символів у маршруті в кінці шляху. У наступному прикладі метод `findAll()` буде виконано для будь-якого маршруту, який починається з `abcd/`, незалежно від кількості символів, що йдуть після нього.

```
@Get('abcd/*')
findAll() {
  return 'This route uses a wildcard';
}
```

Routing

Код статусу за замовчуванням для відповідей завжди дорівнює 200, за винятком **POST**-запитів, для яких за замовчуванням встановлено значення 201. Ви можете легко змінити цю поведінку, використовуючи декоратор `@HttpCode(...)` на рівні обробника.

```
@Post()
@HttpCode(204)
create() {
    return 'This action adds a new cat';
}
```

Routing

Щоб вказати власний заголовок відповіді, можна використовувати декоратор `@Header()` або специфічний для бібліотеки об'єкт відповіді (і викликати `res.header()` безпосередньо).

```
@Post()  
@Header('Cache-Control', 'no-store')  
create() {  
    return 'This action adds a new cat';  
}
```

Routing

Щоб перенаправити відповідь на певну URL-адресу, можна використовувати декоратор `@Redirect()` або специфічний для бібліотеки об'єкт відповіді (і викликати `res.redirect()` безпосередньо).

`@Redirect()` приймає два аргументи: `url` та `statusCode`, обидва необов'язкові.

Якщо пропустити, значення `statusCode` за замовчуванням дорівнює `302` (Found)

```
@Get()  
@Redirect('https://nestjs.com', 301)
```

Повернуті значення перевизначають будь-які аргументи, передані декоратору `@Redirect()`. Наприклад:

```
@Get('docs')  
@Redirect('https://docs.nestjs.com', 302)  
getDocs(@Query('version') version) {  
  if (version && version === '5') {  
    return { url: 'https://docs.nestjs.com/v5/' };  
  }  
}
```

Routing

```
@Get('/:id')
findOne(@Param('id') id: string): string {
  return `This action returns a #${id} cat`;
}
```

```
@Get()
async findAll(
  @Query('age') age: number,
  @Query('breed') breed: string
) {
  return `This action returns all cats filtered by age:
  ${age} and breed: ${breed}`;
}
```

Routing

```
export class CreateCatDto {  
  name: string;  
  age: number;  
  breed: string;  
}
```

```
@Post()  
async create(@Body() createCatDto: CreateCatDto) {  
  return 'This action adds a new cat';  
}
```

Providers

Провайдери є ключовою концепцією в Nest. Багато базових класів Nest, такі як сервіси, репозиторії, фабрики та помічники, можна розглядати як провайдери. Ключова ідея провайдера полягає в тому, що його можна впроваджувати як залежність, що дозволяє об'єктам формувати різні зв'язки один з одним. Відповідальність за "підключення" цих об'єктів значною мірою виконується системою виконання Nest.

```
@Injectable()
export class CatsService {
  private readonly cats: Cat[] = [];

  create(cat: Cat) {
    this.cats.push(cat);
  }

  findAll(): Cat[] {
    return this.cats;
  }
}
```

```
@Controller('cats')
export class CatsController {
  constructor(private catsService: CatsService) {}

  @Post()
  async create(@Body() createCatDto: CreateCatDto) {
    this.catsService.create(createCatDto);
  }

  @Get()
  async findAll(): Promise<Cat[]> {
    return this.catsService.findAll();
  }
}
```

Providers

```
@Injectable()
export class CatsService {
  @Inject(CatsRepository)
  private readonly repo: CatsRepository;
}
```

```
import { Injectable, Optional, Inject } from '@nestjs/common';
```

```
@Injectable()
export class HttpService<T> {
  constructor(
    @Optional() @Inject('HTTP_OPTIONS') private httpClient: T
  ) {}
}
```

Provider scope

DEFAULT - A single instance of the provider is shared across the entire application. The instance lifetime is tied directly to the application lifecycle. Once the application has bootstrapped, all singleton providers have been instantiated. Singleton scope is used by default.

REQUEST - A new instance of the provider is created exclusively for each incoming request. The instance is garbage-collected after the request has completed processing.

TRANSIENT - Transient providers are not shared across consumers. Each consumer that injects a transient provider will receive a new, dedicated instance.

```
@Injectable({ scope: Scope.REQUEST })  
export class UserContext {  
  userId: string;  
}
```

Modules

Модуль — це клас, анотований декоратором `@Module()`. Цей декоратор надає метадані, які Nest використовує для ефективної організації та керування структурою програми.

Кожна програма **Nest** має принаймні один модуль, кореневий модуль, який служить відправною точкою для Nest для побудови графа програми. Цей граф — це внутрішня структура, яку Nest використовує для вирішення зв'язків та залежностей між модулями та провайдерами. Хоча невеликі програми можуть мати лише кореневий модуль, це зазвичай не так. Модулі наполегливо рекомендуються як ефективний спосіб організації ваших компонентів. Для більшості програм у вас, ймовірно, буде кілька модулів, кожен з яких інкапсулює тісно пов'язаний набір можливостей.

Modules

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
    }),
    MongooseModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (configService: ConfigService) => ({
        uri: configService.get<string>('MONGO_ADDRESS'),
      }),
    }),
    GroupModule,
    StudentModule,
  ],
  controllers: [HealthController],
})
export class AppModule {}
```

```
@Module({
  imports: [
    MongooseModule.forFeature([
      { name: Group.name, schema: GroupSchema }
    ]),
  ],
  controllers: [GroupController],
  providers: [GroupService, GroupRepository],
  exports: [GroupService],
})
export class GroupModule {}
```

Modules

Декоратор `@Module()` приймає один об'єкт з властивостями, що описують модуль:

providers – провайдери, екземпляри яких будуть створені Nest-інжектором і які можуть бути спільними принаймні в цьому модулі

controllers – набір контролерів, визначених у цьому модулі, екземпляри яких мають бути створені

imports – список імпортованих модулів, які експортують провайдерів, необхідних у цьому модулі

exports – підмножина провайдерів, що надаються цим модулем і повинні бути доступні в інших модулях, які імпортують цей модуль. Ви можете використовувати або самого провайдера, або лише його токен (provide value)

Модуль інкапсулює провайдерів за замовчуванням, тобто ви можете вставляти лише провайдерів, які є частиною поточного модуля або явно експортовані з інших імпортованих модулів. Експортовані провайдери з модуля по суті служать публічним інтерфейсом або API модуля.

Global modules

```
import { Module, Global } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Global()
@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}
```

Value providers

Синтаксис `useValue` корисний для введення константного значення, розміщення зовнішньої бібліотеки в контейнері Nest або заміни реальної реалізації фіктивним об'єктом. Припустимо, ви хочете змусити Nest використовувати фіктивний `CatsService` для цілей тестування.

```
import { CatsService } from './cats.service';

const mockCatsService = { /* mock implementation */ };

@Module({
  imports: [CatsModule],
  providers: [
    {
      provide: CatsService,
      useValue: mockCatsService,
    },
  ],
})
export class AppModule {}
```

Non-class-based provider tokens

```
import { connection } from './connection';
```

```
@Module({  
  providers: [  
    {  
      provide: 'CONNECTION',  
      useValue: connection,  
    },  
  ],  
})  
export class AppModule {}
```

```
@Injectable()  
export class CatsRepository {  
  constructor(@Inject('CONNECTION') connection: Connection) {}  
}
```

Class providers

```
const configServiceProvider = {  
  provide: ConfigService,  
  useClass:  
    process.env.NODE_ENV === 'development'  
      ? DevelopmentConfigService  
      : ProductionConfigService,  
};  
  
@Module({  
  providers: [configServiceProvider],  
})  
export class AppModule {}
```

Dynamic modules

Динамічні модулі в Nest дозволяють створювати модулі, які можна налаштовувати під час виконання. Це особливо корисно, коли вам потрібно забезпечити гнучкі, настроювані модулі, де провайдери можна створювати на основі певних опцій або конфігурацій.

```
@Module({
  providers: [Connection],
  exports: [Connection],
})
export class DatabaseModule {
  static forRoot(entities = [], options?): DynamicModule {
    const providers = createDatabaseProviders(options, entities);
    return {
      module: DatabaseModule,
      providers: providers,
      exports: providers,
    };
  }
}

@Module({
  imports: [DatabaseModule.forRoot([User])],
})
export class AppModule {}
```

Dynamic modules

```
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { MongooseModule } from '@nestjs/mongoose';
import { HealthController } from '../health/health.controller';
import { GroupModule } from '../group/group.module';
import { StudentModule } from '../student/student.module';
```

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
    }),
    MongooseModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (configService: ConfigService) => ({
        uri: configService.get<string>('MONGO_ADDRESS'),
      }),
    }),
    GroupModule,
    StudentModule,
  ],
  controllers: [HealthController],
})
export class AppModule {}
```

Повертає конфігурацію, яка буде
передана в `mongoose.connect()`



Middleware

```
import { Injectable, NestMiddleware } from '@nestjs/common';  
import { Request, Response, NextFunction } from 'express';
```

```
@Injectable()  
export class LoggerMiddleware implements NestMiddleware {  
  use(req: Request, res: Response, next: NextFunction) {  
    console.log('Request...');  
    next();  
  }  
}
```

```
import { Module, NestModule, MiddlewareConsumer } from '@nestjs/common';  
import { LoggerMiddleware } from '../common/middleware/logger.middleware';  
import { CatsModule } from '../cats/cats.module';
```

```
@Module({  
  imports: [CatsModule],  
})  
export class AppModule implements NestModule {  
  configure(consumer: MiddlewareConsumer) {  
    consumer  
      .apply(LoggerMiddleware)  
      .forRoutes('cats');  
  }  
}
```

Middleware

```
import {
  Module,
  NestModule,
  RequestMethod,
  MiddlewareConsumer,
} from '@nestjs/common';
import { LoggerMiddleware } from
'./common/middleware/logger.middleware';
import { CatsModule } from './cats/cats.module';

@Module({
  imports: [CatsModule],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer
      .apply(LoggerMiddleware)
      .forRoutes({ path: 'cats', method: RequestMethod.GET });
  }
}
```

Middleware

```
consumer.apply(cors(), helmet(), logger).forRoutes(CatsController);
```

```
consumer
  .apply(LoggerMiddleware)
  .exclude(
    { path: 'cats', method: RequestMethod.GET },
    { path: 'cats', method: RequestMethod.POST },
    'cats/{*splat}',
  )
  .forRoutes(CatsController);
```

```
import { Request, Response, NextFunction } from 'express';
export function logger(
  req: Request, res: Response, next: NextFunction
) {
  console.log(`Request...`);
  next();
};
```

Exception filters

Nest має вбудований **exceptions layer**, який відповідає за обробку всіх необроблених **exceptions** у застосунку. Коли **exception** не обробляється кодом вашого застосунку, він перехоплюється цим рівнем, який потім автоматично надсилає відповідну зручну для користувача відповідь.

З коробки ця дія виконується вбудованим **global exception filter**, який обробляє **exceptions** типу **HttpException** (та його підкласів). Коли виняток не розпізнано (не є ні **HttpException**, ні класом, що успадковується від **HttpException**), вбудований фільтр винятків генерує таку відповідь JSON за замовчуванням:

```
{  
  "statusCode": 500,  
  "message": "Internal server error"  
}
```

Exception filters

```
@Get()
async findAll() {
  throw new HttpException('Forbidden', HttpStatus.FORBIDDEN);
}

{
  "statusCode": 403,
  "message": "Forbidden"
}
```

```
@Get()
async findAll() {
  try {
    await this.service.findAll()
  } catch (error) {
    throw new HttpException({
      status: HttpStatus.FORBIDDEN,
      error: 'This is a custom message',
    }, HttpStatus.FORBIDDEN, {
      cause: error
    });
  }
}
```

Exception filters

У багатьох випадках вам не потрібно буде писати власні `exceptions`, і ви можете використовувати вбудований `exception` Nest HTTP. Якщо вам потрібно створити власні `exceptions`, гарною практикою буде створити власну ієрархію `exceptions`, де ваші власні `exceptions` успадковуються від базового класу `HttpException`. Завдяки такому підходу Nest розпізнаватиме ваші `exceptions` та автоматично оброблятиме відповіді на помилки.

```
export class ForbiddenException extends HttpException {  
  constructor() {  
    super('Forbidden', HttpStatus.FORBIDDEN);  
  }  
}  
  
@Get()  
async findAll() {  
  throw new ForbiddenException();  
}
```

Built-in HTTP exceptions

BadRequestException
UnauthorizedException
NotFoundException
ForbiddenException
NotAcceptableException
RequestTimeoutException
ConflictException
GoneException
HttpVersionNotSupportedException
PayloadTooLargeException
UnsupportedMediaTypeException
UnprocessableEntityException
InternalServerErrorException
NotImplementedException
ImATeapotException
MethodNotAllowedException
BadGatewayException
ServiceUnavailableException
GatewayTimeoutException
PreconditionFailedException

Exception filters

```
import { ExceptionFilter, Catch, ArgumentsHost, HttpException } from '@nestjs/common';
import { Request, Response } from 'express';
```

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();
    const status = exception.getStatus();

    response
      .status(status)
      .json({
        statusCode: status,
        timestamp: new Date().toISOString(),
        path: request.url,
      });
  }
}
```

```
@Post()
@UseFilters(new HttpExceptionFilter())
async create(@Body() createCatDto: CreateCatDto) {
  throw new ForbiddenException();
}
```

Exception filters

```
@Post()
@UseFilters(new HttpExceptionFilter())
async create(@Body() createCatDto: CreateCatDto) {
  throw new ForbiddenException();
}
```

```
@Controller()
@UseFilters(new HttpExceptionFilter())
export class CatsController {}
```

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalFilters(new HttpExceptionFilter());
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Стосовно DI, глобальні фільтри, зареєстровані ззовні будь-якого модуля (за допомогою `useGlobalFilters()`), не можуть впроваджувати залежності, оскільки це робиться поза контекстом будь-якого модуля. Щоб вирішити цю проблему, можна зареєструвати фільтр глобальної області видимості безпосередньо з будь-якого модуля, використовуючи таку конструкцію:

```
import { Module } from '@nestjs/common';
import { APP_FILTER } from '@nestjs/core';

@Module({
  providers: [{
    provide: APP_FILTER,
    useClass: HttpExceptionFilter,
  }],
})
export class AppModule {}
```



Pipe — це клас, анотований декоратором `@Injectable()`, який реалізує інтерфейс `PipeTransform`.

Pipes мають два типові випадки використання:

- Трансформація: перетворення вхідних даних у потрібну форму (наприклад, з рядка на ціле число)
- Валідація: оцінка вхідних даних і, якщо вони дійсні, просто передача їх без змін; інакше, викидання exception

```
@Get('/:id')  
findOne(@Param('id', ParseIntPipe) id: number) {}
```

"42" → число 42

```
@Post()  
create(@Body() dto: CreateUserDto) {}
```

Якщо дані невалідні → помилка ще **до** контролера.

Pipes

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
  } ),  
);
```

`whitelist` — викидаємо зайві поля

`forbidNonWhitelisted` — забороняємо сміття

`transform` — приводить вхідні дані до типів з DTO.

```
export class CreateUserDto {  
  @IsEmail()  
  email: string;  
  
  @IsInt()  
  age: number;  
}
```

- контролер навіть не буде викликатись

```
{ "email": "nope", "age": "abc" }
```

Pipes

Built-in pipes

Nest comes with several pipes available out-of-the-box:

- ValidationPipe
- ParseIntPipe
- ParseFloatPipe
- ParseBoolPipe
- ParseArrayPipe
- ParseUUIDPipe
- ParseEnumPipe
- DefaultValuePipe
- ParseFilePipe
- ParseDatePipe

They're exported from the [@nestjs/common](#) package.

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.setGlobalPrefix('api');
  app.useGlobalPipes(
    new ValidationPipe({
      transform: true,
      whitelist: true,
    }),
  );

  const configService = app.get(ConfigService);
  const host = configService.get<string>('HOST', 'localhost');
  const port = configService.get<number>('PORT', 3000);

  await app.listen(port, host);
  Logger.log(banner);
  Logger.log(`🚀 Example app listening on port: ${host}:${port}`);
  app.enableCors();
}

bootstrap();
```

```
import { Module } from '@nestjs/common';
import { StudentService } from './student.service';
import { StudentController } from './student.controller';
import { MongooseModule } from '@nestjs/mongoose';
import { Student, StudentSchema } from './student.schema';
import { StudentRepository } from './student.repository';
import { GroupModule } from 'src/group/group.module';
```

```
@Module({
  imports: [
    MongooseModule.forFeature([
      {
        name: Student.name,
        schema: StudentSchema,
      },
    ]),
    GroupModule,
  ],
  providers: [StudentService, StudentRepository],
  controllers: [StudentController],
})
export class StudentModule {}
```

Mongo model

```
@Schema({ _id: false })
export class StudentAddress {
  @Prop({ required: true }) country: string;
  @Prop({ required: true }) town: string;
  @Prop({ required: true }) addressString: string;
}
export const StudentAddressSchema = SchemaFactory.createForClass(StudentAddress);

@Schema({ timestamps: true })
export class Student {
  @Prop({ required: true }) name: string;
  @Prop({ required: true }) surname: string;
  @Prop({ required: true, type: Types.ObjectId, ref: 'Group' }) groupId: Types.ObjectId;
  @Prop({
    required: true,
    type: Date,
    validate: {
      validator: (value: Date) => value <= new Date(),
      message: (props: { value: Date }) =>
        `Birth date cannot be in the future. Provided date: ${props.value}`,
    },
  }) birthDate: Date;
  @Prop({ type: [String] }) phoneNumbers?: string[];
  @Prop({ type: StudentAddressSchema }) address?: StudentAddress;
}

export const StudentSchema = SchemaFactory.createForClass(Student);
```

```
@Injectable()
export class StudentRepository {
  constructor(
    @InjectModel(Student.name) private readonly model: Model<StudentDocument>,
  ) {}

  async get(id: string): Promise<Student | null> {
    return this.model.findById(id).lean();
  }

  async create(data: StudentCreateDto): Promise<string> {
    const student = await this.model.create(data);
    return student.id;
  }

  async update(id: string, data: StudentUpdateDto): Promise<void> {
    await this.model.updateOne({ _id: id }, { $set: data });
  }

  async delete(id: string): Promise<void> {
    await this.model.findByIdAndDelete(id);
  }
}
```

Tests

```
let studentRepository: StudentRepository;
let studentModel: Model<StudentDocument>;
let groupModel: Model<Group>;

beforeAll(async () => {
  const uri = await startMongoContainer();
  const module: TestingModule = await Test.createTestingModule({
    imports: [
      mongooseModule.forRoot(uri),
      mongooseModule.forFeature([
        { name: Student.name, schema: StudentSchema },
        { name: Group.name, schema: GroupSchema },
      ]),
    ],
    providers: [StudentRepository],
  }).compile();

  studentRepository = module.get<StudentRepository>(StudentRepository);
  studentModel = module.get<Model<StudentDocument>>(
    getModelToken(Student.name),
  );
  groupModel = module.get<Model<Group>>(getModelToken(Group.name));
});
```

Виклик зовнішніх API

1. `npm install @nestjs/axios axios`

2.

```
import { Module } from '@nestjs/common';
import { HttpModule } from '@nestjs/axios';
import { CatsService } from './cats.service';
```

```
@Module({
  imports: [HttpModule],
  providers: [CatsService],
})
export class CatsModule {}
```

3.

```
import { Injectable } from '@nestjs/common';
import { HttpService } from '@nestjs/axios';
import { firstValueFrom } from 'rxjs';
```

```
@Injectable()
export class CatsService {
  constructor(private readonly httpService: HttpService) {}

  async findAll(): Promise<Cat[]> {
    const { data } = await firstValueFrom(
      this.httpService.get<Cat[]>('http://localhost:3000/cats'),
    );
    return data;
  }
}
```

ConfigModule

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
```

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
    }),
  ],
})
export class AppModule {}
```

файл .env:

API_BASE_URL=http://localhost:3000

```
@Injectable()
export class CatsService {
  constructor(
    private readonly httpService: HttpService,
    private readonly configService: ConfigService,
  ) {}

  async findAll(): Promise<Cat[]> {
    const url = this.configService.get<string>('API_BASE_URL');

    const { data } = await firstValueFrom(
      this.httpService.get<Cat[]>(`${url}/cats`),
    );
    return data;
  }
}
```



Дякуємо за увагу!

<https://github.com/ArtemPetrenkoEduardovich/nodejs-mongo-sample>